GPU Biometric Matching

Simon Barras

School of Computer and Communication Sciences

Semester Project

March 4, 2026

Responsible
Prof. Serge Vaudenay
EPFL / LASEC

Abstract

Abastract....

Contents

List of Figures **4**

List of Tables **4**

Acronyms **5**

1 Introduction **6**

2 Analysis **7**
2.1 The Miura Pipeline 7
2.1.1 Previous Work 7
2.1.2 1-to-N Matching 8
2.2 Graphics Processing Units 8
2.2.1 Architecture 8
2.2.2 Manufacturers and Programming Models 10
2.2.3 GPU Programming 10

3 Conception **12**

4 Realisation **13**

5 Evaluation **14**

6 Discussion & Future Work **15**

7 Conclusion **16**

References **17**

A SYCL Matrix Multiplication Example **18**

List of Figures

1 The flow of the optimized pipeline, adapted from Lara Sofie Lenz's report [3] 7
2 A GPU architecture schema [2] 9
3 Memory hierarchy in a PC with a discrete GPU 11

List of Tables

Acronyms

CPU Central Procesing Unit.

CUDA Computaional Unified Device Architecture.

FFT Fast Fourier Transform.

GPU Graphics Processing Unit.

HIP Heterogeneous-computing Interface for Portability.

OpenCL Open Computing Language.

RAM Random Access Memory.

SIMD Single Instruction Multiple Data.

SIMT Single Instruction Multiple Threads.

SISD Single Instruction Single Data.

SM Streaming Microprocessor.

SYCL System-wide Compute Language.

TPU Tensor Processing Unit.

VRAM Video Random Access Memory.

1 Introduction

2 Analysis

This project focuses on optimizing the matching component of Miura's pipeline [4] to support **1-to-N** matching. The primary objective is to leverage a GPU to ensure efficient processing times even when searching against a large database of users.

2.1 The Miura Pipeline

The Miura pipeline, introduced in [4], is a method that processes a single image of a finger to extract vein patterns. It then computes the distance between these extracted veins and a reference pattern. The method is recognized for its high performance and accuracy, demonstrating the viability of using finger vein patterns as a biometric attribute for authentication.

2.1.1 Previous Work

Since its publication in 2004 [4], several research projects at the EPFL LASEC lab have built upon the Miura pipeline, specifically exploring its use for patient identification and authentication in hospital environments.

Optimized Pipeline

In 2024, Lara Sofie Lenz developed an optimized implementation of the Miura pipeline [3]. This C++ implementation is approximately 10 times faster than the original Python version for **1-to-1** matching. The flow of this optimized pipeline is illustrated in Figure 1.

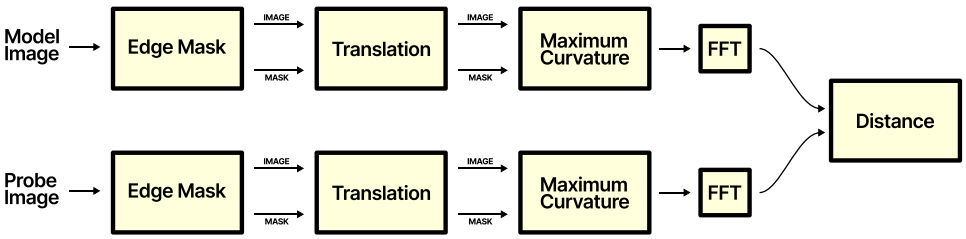

Figure 1: The flow of the optimized pipeline, adapted from Lara Sofie Lenz's report [3]

This optimized implementation serves as the foundation for this project. While the extraction stage of the pipeline remains unchanged, the matching stage—currently designed for 1-to-1 matching—will be modified to perform 1-to-N matching using a GPU.

Mathematical Background

As shown in Figure 1, the optimized pipeline utilizes a Fast Fourier Transform (FFT) to compute the cross-correlation between extracted veins and the reference pattern. A detailed mathematical explanation of using FFT for signal cross-correlation can be found in Vincent Luke Ventura's project on Biometric Homomorphic Matching [6].

2.1.2 1-to-N Matching

Previous research at LASEC regarding the biometric identification methods mentioned in Section 2.1.1 has demonstrated strong performance (under 1 second) for **1-to-1** matching. While 1-to-1 matching is inherently faster, it requires users to provide an ID before authentication. In contrast, 1-to-N matching is essential during the registration phase to ensure a user is not already present in the database.

The objective is to compute the cross-correlation between the extracted veins and all reference patterns in the database simultaneously. This requires a shift in architectural design from the CPU, a general-purpose processor, to the GPU, which is specialized hardware designed for massive parallel computation.

The scope of this project is limited to modifying the matching stage of Lenz's optimized pipeline [3]. The extraction logic will remain intact, while the matching component will be redesigned to leverage the GPU for simultaneous cross-correlation across the entire database.

2.2 Graphics Processing Units

Graphics Processing Unit (GPU)s are specialized hardware components designed for efficient parallel computation. They typically function as co-processors to a CPU, accelerating specific data-parallel applications.

2.2.1 Architecture

The standard architecture for a computer's main processor is the CPU. It is designed for a broad range of tasks and is optimized for low-latency, single-threaded (or few-threaded) performance. It is generally categorized as Single Instruction Single Data, although modern multi-core CPUs do support levels of parallelism.

Conversely, the GPU is built for high-throughput parallel tasks and is classified as Single Instruction Multiple Data because it can physically execute the same instruction across multiple data points simultaneously.

Note that Nvidia refers to their architecture as Single Instruction Multiple Threads to emphasize that while threads execute the same instruction.

A GPU typically consists of several Graphics Processor Cluster (GPC), each containing multiple Streaming Microprocessor (SM). These SMs contain multiple cores, and each core executes a thread. Threads are logically organized into blocks. A key distinction of the GPU is that all threads within a block execute **the same instruction** at the same time.

As illustrated in Figure 2, GPUs utilize a different memory hierarchy than CPUs, prioritizing high throughput over low latency. The L2 cache is shared across all SMs, and while global memory is accessible by all threads, it has higher latency compared to the shared memory available to threads within a specific block.

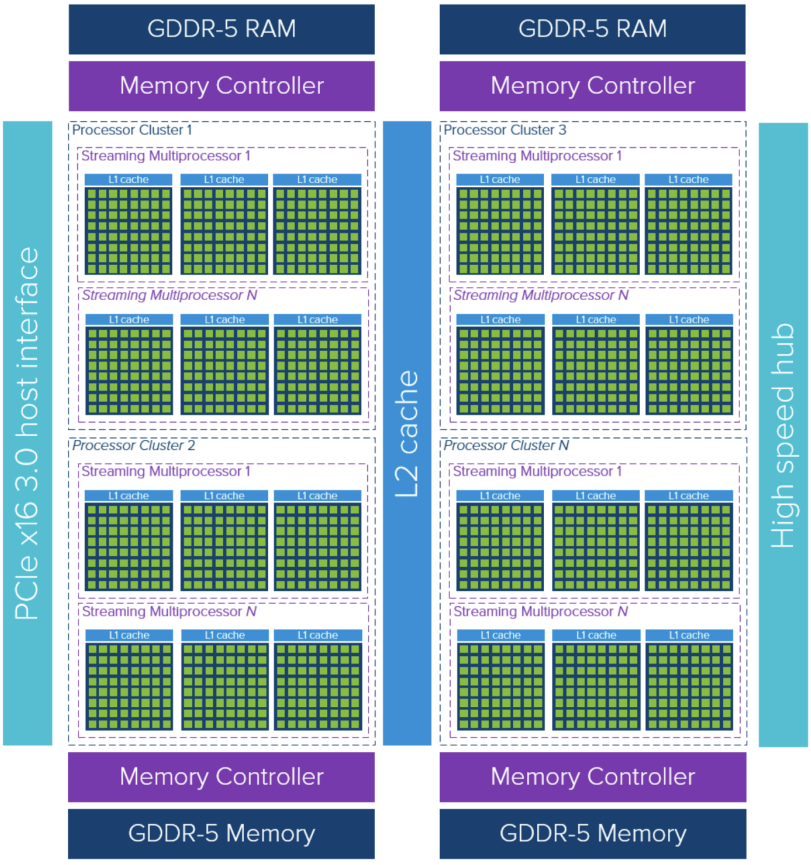

Figure 2: A GPU architecture schema [2]

While different manufacturers have unique thread organizations and memory hierarchies, the underlying principle remains the same. The pri-

mary differences usually lie in core counts and memory bandwidth. Alternative architectures, such as Apple Silicon, employ a unified memory architecture where the CPU and GPU share a single memory pool, allowing both to access data without copying it.

2.2.2 Manufacturers and Programming Models

In 2026, the two dominant GPU architecture designers are Nvidia and AMD. Emerging competitors include Intel, with their Intel Arc series, and Apple, which integrates GPUs into their Silicon chips. Additionally, Google's Tensor Processing Unit (TPU) isn't a GPU but it is a specialized hardware for machine learning.

Utilizing these devices requires a programming model to offload data and computations. CUDA is a proprietary model developed by Nvidia and is exclusive to their hardware. AMD's HIP is another programming model designed to allow code to run on both Nvidia and AMD hardware. OpenCL is an open standard supporting a wide range of devices from various manufacturers.

The choice of programming model is critical for performance and portability. While OpenCL offers broad compatibility, its performance is not always optimal, the coding style is dated, and some manufacturers have reduced their support for it.

SYCL [5] is an abstraction layer that enables code execution across different hardware manufacturers. Unlike OpenCL, SYCL provides a high-level, modern C++ style that compiles to the target architecture's native model (such as CUDA or HIP). This is highly beneficial during development, as the specific end-device may not yet be known.

Performance remains a key consideration. According to "Comparing Performance and Portability Between CUDA and SYCL for Protein Database Search on NVIDIA, AMD, and Intel GPUs" [1], SYCL achieves minimal overhead performance compared naive model like CUDA or HIP.

For this project, SYCL will be used to implement the 1-to-N matching. It provides broad compatibility and simplifies development on workstations that may lack a specific GPU brand.

2.2.3 GPU Programming

Since the GPU acts as a co-processor, code is executed on the CPU by default. To utilize the GPU, developers must explicitly offload computations and data. Figure 3 illustrates the memory organization of a system with a

GPU. Data in the host memory (RAM) is not directly accessible by the GPU; it must be copied to the device memory (VRAM) before processing.

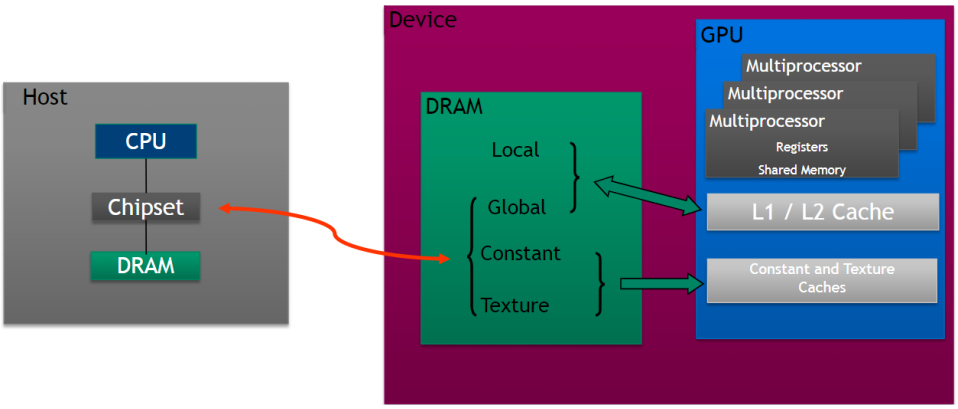

Figure 3: Memory hierarchy in a PC with a discrete GPU

The standard workflow for GPU acceleration, regardless of the programming model, is as follows:

1. Allocate variables in the host memory.
2. Allocate corresponding variables in the device memory.
3. Copy data from the host memory to the device memory.
4. Launch the kernel on the device.
5. Copy the results from the device memory back to the host memory.
6. Free the device memory.

Appendix A provides an example of SYCL code for matrix multiplication following these steps.

3 Conception

4 Realisation

5 Evaluation

6 Discussion & Future Work

7 Conclusion

References

[1] Manuel Costanzo, Enzo Rucci, Carlos García-Sánchez, Marcelo Naiouf, and Manuel Prieto-Matías. Comparing performance and portability between cuda and sycl for protein database search on nvidia, amd, and intel gpus. In *2023 IEEE 35th International Symposium on Computer Architecture and High Performance Computing (SBAC-PAD)*, pages 141–148, 2023. doi:10.1109/SBAC-PAD59825.2023.00023.

[2] Niels Hagoort. Exploring the gpu architecture. Accessed: 2024-02-26. URL: https://www.vmware.com/docs/exploring-the-gpu-architecture.

[3] Lara Sofie Lenz. Finger vein biometric matching. Semester project thesis, EPFL, June 2024.

[4] Naoto Miura, Akio Nagasaka, and Takafumi Miyatake. Feature extraction of finger vein patterns based on iterative line tracking and its application to personal identification. *Syst. Comput. Japan*, 35(7):61–71, jun 2004.

[5] The Khronos SYCL Working Group. SYCL 2020 Specification (revision 10). https://www.khronos.org/registry/SYCL/specs/sycl-2020/html/sycl-2020.html, 2024. Accessed: 2026-02-25.

[6] Vincent Luke Ventura. Biometric homomorphic matching. Semester project thesis, EPFL, January 2026.

A SYCL Matrix Multiplication Example

This appendix provides a SYCL implementation of a matrix multiplication kernel to illustrate the GPU programming workflow described in Section 2.2.3. The code demonstrates how to orchestrate parallel computation on a device and includes annotations for each phase of the execution.

1. **Matrix Dimension Initialization** Defining the matrix dimensions and calculating the required memory sizes in bytes.

```
int M = 512, K = 512, N = 512;
size_t sizeA = M * K * sizeof(float);
size_t sizeB = K * N * sizeof(float);
size_t sizeC = M * N * sizeof(float);
```

2. **Host Memory Allocation and Initialization** Allocating memory on the CPU (host) and populating the input matrices with initial data.

```
// Allocate host memory
float* h_A = static_cast<float*>(malloc(sizeA));
float* h_B = static_cast<float*>(malloc(sizeB));
float* h_C = static_cast<float*>(malloc(sizeC));

// Initialize matrices with dummy data
for (int i = 0; i < M * K; i++) h_A[i] = 1.0f;
for (int i = 0; i < K * N; i++) h_B[i] = 2.0f;
```

3. **SYCL Queue Creation** The queue serves as the primary interface for submitting command groups to the accelerator device.

```
sycl::queue q;

// Optional - Output the selected device name
std::cout << "Running on: "
          << q.get_device().get_info<sycl::info::device::name>()
          << std::endl;
```

4. **Device Allocation and Data Transfer** Allocating memory directly on the GPU (device) and transferring data from the host.

```
// Allocate device memory using Unified Shared Memory (USM) device
    pointers
float* d_A = sycl::malloc_device<float>(M * K, q);
float* d_B = sycl::malloc_device<float>(K * N, q);
float* d_C = sycl::malloc_device<float>(M * N, q);

// Explicitly copy data to device memory
q.memcpy(d_A, h_A, sizeA);
q.memcpy(d_B, h_B, sizeB).wait(); // Ensure transfer is complete
    before proceeding
```

5. **Kernel Dispatch** SYCL offers several methods for launching kernels on the device:

   (a) **Anonymous Lambda Function:** A concise method capturing variables by value using `[=]`.

```
q.parallel_for(sycl::range<2>(M, N), [=](sycl::id<2> index) {
    int row = index[0];
    int col = index[1];
    float sum = 0.0f;
    for (int i = 0; i < K; ++i) {
        sum += d_A[row * K + i] * d_B[i * N + col];
    }
    d_C[row * N + col] = sum;
}).wait();
```

   (b) **Named Lambda:** Wrapping the kernel in a command group handler (`submit`). This is particularly useful for debugging and profiling tools.

```
q.submit([&](sycl::handler& h) {
    h.parallel_for<class MatMul>(sycl::range<2>(M, N),
                                 [=](sycl::id<2> index) {
        int row = index[0];
        int col = index[1];
        float sum = 0.0f;
        for (int i = 0; i < K; ++i) {
            sum += d_A[row * K + i] * d_B[i * N + col];
        }
        d_C[row * N + col] = sum;
    });
}).wait();
```

   (c) **Functor Class:** Defining a function object to encapsulate the kernel logic, providing a clean separation between host and device code.

```
struct MatrixMulFunctor {
    float *A, *B, *C;
    int M, K, N;

    // Constructor to capture host-side parameters
    MatrixMulFunctor(float* a, float* b, float* c,
                     int m, int k, int n)
        : A(a), B(b), C(c), M(m), K(k), N(n) {}

    // The call operator defines the kernel entry point
    void operator()(sycl::id<2> index) const {
        int row = index[0];
        int col = index[1];
        if (row < M && col < N) {
            float sum = 0.0f;
            for (int i = 0; i < K; ++i) {
```

```
                sum += A[row * K + i] * B[i * N + col];
            }
            C[row * N + col] = sum;
        }
    }
};

// Dispatching the kernel using the functor instance
q.submit([&](sycl::handler& h) {
    MatrixMulFunctor functor(d_A, d_B, d_C, M, K, N);
    h.parallel_for(sycl::range<2>(M, N), functor);
}).wait();
```

6. **Result Retrieval** Copying the computed results from device memory back to the host for verification.

```
q.memcpy(h_C, d_C, sizeC).wait();
std::cout << "Success! Element [0,0] is: " << h_C[0]
          << " (Expected: " << K * 2.0f << ")"
          << std::endl;
```

7. **Memory Deallocation** Releasing both device and host memory to prevent leaks.

```
sycl::free(d_A, q);
sycl::free(d_B, q);
sycl::free(d_C, q);
free(h_A);
free(h_B);
free(h_C);
```